



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Métodos para seleccionar elementos del DOM

Presentado a: Instructora Alexandra Guevara
Por Aprendiziz: Cristian David Yalanda
Ficha: 3312932
Grupo: ADSO

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **29** de **08** del año **2026**



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Tabla de Contenido

Contenido

Actividad – 1 Investigación de conceptos.....	3
Enunciado	3
Solución 1	4
Solución 2	5
Solución 3	7
Solución 4	9
Actividad – 2 Comparación de métodos del DOM	10
Enunciado	10
Solución 1	10
Solución 2	11
Solución 3	12
Solución 4	13
Solución 5	14
Actividad – 3 Investigación sobre selectores CSS	15
Enunciado	15
Solución	15
Actividad – 4 Laboratorio de experimentación	16
Actividad – 5 Preguntas de análisis	16
Enunciado	16
Solución	17
Actividad – 6 Reto investigativo	17
Enunciado	17
Solución	18
Actividad – 7 Reto práctico	18



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Actividad – 1 Investigación de conceptos

Enunciado

Investiga y explica con tus propias palabras los siguientes métodos:

- `document.getElementsByClassName()` 1
- `document.getElementsByTagName()` 2
- `document.querySelector()` 3
- `document.querySelectorAll()` 4

Para cada método debes investigar:

1. ¿Para qué sirve?
2. ¿Qué tipo de elementos permite seleccionar?
3. ¿Qué recibe como parámetro?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

4. ¿Puede seleccionar uno o varios elementos?
5. ¿Qué devuelve?
6. ¿Cómo se puede acceder a los elementos encontrados?
7. ¿Cómo se pueden recorrer los elementos?
8. ¿Qué ocurre cuando no encuentra ningún elemento?
9. ¿Qué ventajas tiene?
10. ¿Qué limitaciones tiene?

Solución 1

- `document.getElementsByClassName()`

¿Para qué sirve?

Sirve para buscar elementos HTML que tengan una determinada clase

¿Qué tipo de elementos permite seleccionar?

Puede seleccionar cualquier elemento HTML que tenga la clase indicada, por ejemplo:

```
<p class="texto">
```

```
<div class="texto">
```

```
<button class="texto">
```

```
<h1 class="texto">
```

¿Qué recibe como parámetro?

Recibe el nombre de una o varias clases como texto

¿Puede seleccionar uno o varios elementos?

Puede seleccionar uno o varios elementos. Aunque solamente exista un elemento con esa clase, el método devuelve una colección

¿Qué devuelve?

Devuelve una `HTMLCollection`. Es parecido a un arreglo porque podemos acceder a los elementos mediante posiciones:

```
elementos [0]
```

```
elementos [1]
```

¿Cómo se puede acceder a los elementos encontrados?

Se puede utilizar su posición:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

elementos [0]

También se puede utilizar item():

elementos.item(0)

¿Cómo se pueden recorrer los elementos?

Una forma sencilla es utilizando un for:

```
for (let i = 0; i < elementos.length; i++) {  
    console.log(elementos[i]) }
```

length indica cuántos elementos fueron encontrados

¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve una HTMLCollection vacía

¿Qué ventajas tiene?

Es sencillo de utilizar

Es específico para buscar por clases

Puede encontrar varios elementos

Permite acceder fácilmente mediante posiciones

La colección se actualiza automáticamente cuando cambia el DOM

¿Qué limitaciones tiene?

Solo permite buscar principalmente por clases.

Devuelve una HTMLCollection, no un arreglo normal.

Al ser una colección dinámica, hay que tener cuidado al modificar los elementos mientras se recorren.

Solución 2

- document.getElementsByTagName()

¿Para qué sirve?

Sirve para buscar elementos utilizando el nombre de la etiqueta HTML

¿Qué tipo de elementos permite seleccionar?

Puede seleccionar cualquier etiqueta HTML:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
document.getElementsByTagName("div")
```

```
document.getElementsByTagName("button")
```

```
document.getElementsByTagName("h1")
```

```
document.getElementsByTagName("input")
```

¿Qué recibe como parámetro?

Recibe el nombre de la etiqueta como texto:

```
document.getElementsByTagName("p")
```

En este caso "p" es el parámetro

¿Puede seleccionar uno o varios elementos?

Puede seleccionar uno o varios elementos. Por ejemplo, si tenemos cinco `<p>`, encontrará los cinco.

¿Qué devuelve?

Devuelve una `HTMLCollection` con los elementos encontrados.

¿Cómo se puede acceder a los elementos encontrados?

Mediante su posición:

```
botones[0]
```

```
botones[1]
```

¿Cómo se pueden recorrer los elementos?

Se pueden recorrer con un `for`:

```
for (let i = 0; i < botones.length; i++) {  
    console.log(botones[i])  
}
```

¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve una `HTMLCollection` vacía.

Por ejemplo, si no hay ningún `<video>`:

```
let videos = document.getElementsByTagName("video")
```

entonces:

```
videos.length
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

será 0

¿Qué ventajas tiene?

- Es fácil de entender
- Es útil cuando necesitamos todos los elementos de una determinada etiqueta
- Puede seleccionar varios elementos
- Permite acceder a los resultados mediante índices

¿Qué limitaciones tiene?

- Su búsqueda se basa principalmente en etiquetas HTML
- No permite hacer búsquedas tan específicas como `querySelector()`
- Devuelve una `HTMLCollection`
- La colección es dinámica, por lo que cambia cuando cambia el DOM

Solución 3

- `document.querySelector()`

¿Para qué sirve?

Sirve para seleccionar el primer elemento que coincida con un selector CSS.

¿Qué tipo de elementos permite seleccionar?

Permite buscar prácticamente cualquier elemento utilizando selectores CSS

¿Qué recibe como parámetro?

Recibe un **selector CSS escrito como texto**.

Por ejemplo:

```
document.querySelector(".boton")
```

Aquí: `".boton"` es el parámetro.

¿Puede seleccionar uno o varios elementos?

Puede encontrar muchos elementos que coincidan, pero solamente devuelve el primero

Por ejemplo, si tenemos:

```
<p class="texto">Uno</p>
```

```
<p class="texto">Dos</p>
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

`<p class="texto">Tres</p>`

y hacemos: `let elemento = document.querySelector(".texto")`

solo obtendremos: `<p class="texto">Uno</p>`

¿Qué devuelve?

Devuelve un Element, es decir, el primer elemento encontrado.

Si no encuentra ninguno, devuelve:

Null

¿Cómo se puede acceder a los elementos encontrados?

Como devuelve directamente un elemento, podemos utilizarlo directamente:

`let titulo = document.querySelector("#titulo")`

`titulo.textContent = "Hola"`

No necesitamos utilizar `[0]` porque ya devuelve el primer elemento.

¿Cómo se pueden recorrer los elementos?

Normalmente no se recorre, porque solo devuelve un elemento. Si necesitamos recorrer varios elementos, debemos utilizar:

`querySelectorAll()`

¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve: null

Por eso hay que tener cuidado de no intentar utilizar propiedades de un elemento que no existe.

¿Qué ventajas tiene?

- Es muy flexible.
- Permite utilizar selectores CSS.
- Puede buscar por ID, clase, etiqueta, atributos, etc.
- Es sencillo para seleccionar un único elemento.
- Devuelve directamente el elemento.

¿Qué limitaciones tiene?

- Solo devuelve el primer elemento que encuentra.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

- Si necesitamos todos los elementos coincidentes, no sirve por sí solo.
- El selector utilizado debe tener una sintaxis CSS válida.

Solución 4

- `document.querySelectorAll()`

¿Para qué sirve?

Sirve para seleccionar todos los elementos que coincidan con uno o varios selectores CSS

¿Qué tipo de elementos permite seleccionar?

Puede seleccionar prácticamente cualquier elemento mediante selectores CSS

¿Qué recibe como parámetro?

Recibe uno o varios selectores CSS como texto

¿Puede seleccionar uno o varios elementos?

Sí puede devolver:

- Ningún elemento.
- Un elemento.
- Muchos elementos

¿Qué devuelve?

Devuelve una NodeList

¿Cómo se puede acceder a los elementos encontrados?

Se puede utilizar la posición:

`botones[0]`

`botones[1]`

¿Cómo se pueden recorrer los elementos?

Podemos utilizar un for:

```
for (let i = 0; i < botones.length; i++) {
```

```
  console.log(botones[i])
```

```
}
```

También existen otras formas de recorrer una NodeList, como `forEach()`

¿Qué ocurre cuando no encuentra ningún elemento?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

No devuelve null.

Devuelve una NodeList vacía

¿Qué ventajas tiene?

- Es muy flexible.
- Permite utilizar selectores CSS.
- Puede seleccionar varios elementos.
- Permite combinar diferentes selectores.
- Es muy útil para trabajar con grupos de elementos.

¿Qué limitaciones tiene?

- La sintaxis del selector debe ser correcta.
- Devuelve una NodeList, no un arreglo normal.
- La lista es estática y no se actualiza automáticamente cuando cambia el DOM.
- Si solo necesitamos un elemento, `querySelector()` es más directo

Actividad – 2 Comparación de métodos del DOM

Enunciado

Investiga las diferencias entre:

- `getElementById()` y `getElementsByClassName()`
- `getElementsByClassName()` y `getElementsByTagName()`
- `querySelector()` y `querySelectorAll()`
- `getElementsByClassName()` y `querySelectorAll()`
- `getElementById()` y `querySelector()`

Explica cuándo utilizarías cada uno.

Solución 1

- `getElementById()` y `getElementsByClassName()`

La diferencia principal es qué buscan y cuántos elementos pueden devolver

Característica	<code>getElementById()</code>	<code>getElementsByClassName()</code>
Busca por	id	class
Puede encontrar	Un elemento	Uno o varios
Devuelve	Un elemento	HTMLCollection
Si no encuentra	null	Colección vacía



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Ejemplo	"titulo"	"boton"
---------	----------	---------

getElementById()

Se utiliza cuando queremos encontrar un elemento específico que tiene un ID

```
let titulo = document.getElementById("titulo")
```

Por ejemplo, si tenemos:

```
<h1 id="titulo">Hola</h1>
```

Lo utilizaría cuando sé exactamente qué elemento necesito y ese elemento tiene un id único

getElementsByClassName()

Se utiliza cuando queremos encontrar varios elementos que tienen la misma clase

```
let botones = document.getElementsByClassName("boton")
```

Por ejemplo:

```
<button class="boton">Guardar</button>
```

```
<button class="boton">Eliminar</button>
```

```
<button class="boton">Editar</button>
```

Aquí obtendríamos los tres botones.

¿Cuándo utilizaría cada uno?

Usaría `getElementById()` cuando necesito trabajar con un único elemento específico.

Usaría `getElementsByClassName()` cuando necesito trabajar con varios elementos que comparten una clase.

Solución 2

- `getElementsByClassName()` y `getElementsByTagName()`

Estos dos métodos son muy parecidos porque los dos pueden devolver varios elementos y devuelven una `HTMLCollection`. La diferencia está en la forma de realizar la búsqueda.

Característica	<code>getElementsByClassName()</code>	<code>getElementsByTagName()</code>
Busca por	Clase	Etiqueta
Ejemplo	"boton"	"button"
Puede devolver	Varios	Varios
Devuelve	<code>HTMLCollection</code>	<code>HTMLCollection</code>



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

getElementsByClassName()

Busca por la clase:

```
let elementos = document.getElementsByClassName("producto")
```

Encontraría elementos como:

```
<div class="producto">Producto 1</div>
```

```
<div class="producto">Producto 2</div>
```

getElementsByTagName()

Busca por la etiqueta HTML:

```
let parrafos = document.getElementsByTagName("p")
```

Encontraría todos los:

```
<p>Texto 1</p>
```

```
<p>Texto 2</p>
```

```
<p>Texto 3</p>
```

¿Cuándo utilizaría cada uno?

Usaría `getElementsByClassName()` cuando me interese una característica común que tienen varios elementos, por ejemplo todos los elementos con la clase "producto".

Usaría `getElementsByTagName()` cuando simplemente necesite todos los elementos de un determinado tipo, por ejemplo todos los `<p>`, `<button>` o `<div>`

Solución 3

- `querySelector()` y `querySelectorAll()`

Aquí la diferencia es muy fácil:

`querySelector()` → devuelve el primero.

`querySelectorAll()` → devuelve todos.

Ambos utilizan selectores CSS. `querySelector()` devuelve el primer elemento coincidente o null, mientras que `querySelectorAll()` devuelve una `NodeList` con todos los elementos coincidentes

Característica	<code>querySelector()</code>	<code>querySelectorAll()</code>
Utiliza	Selectores CSS	Selectores CSS
Devuelve	Primer elemento	Todos los elementos
Resultado	Element	NodeList
Si no encuentra	null	NodeList vacía



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

querySelector()

Por ejemplo:

```
let boton = document.querySelector(".boton")
```

Si existen cinco elementos con la clase boton, solamente seleccionará el primero

querySelectorAll()

```
let botones = document.querySelectorAll(".boton")
```

Si existen cinco elementos con la clase boton, seleccionará los cinco

¿Cuándo utilizaría cada uno?

Usaría `querySelector()` cuando solamente necesito un elemento

Usaría `querySelectorAll()` cuando necesito varios elementos que cumplen una condición

Solución 4

- `getElementsByClassName()` y `querySelectorAll()`

Estos dos pueden parecer iguales porque ambos permiten seleccionar varios elementos por clase, pero funcionan de manera diferente

Característica	<code>getElementsByClassName()</code>	<code>querySelectorAll()</code>
Búsqueda	Por clase	Por selector CSS
Puede devolver	Varios	Varios
Devuelve	HTMLCollection	NodeList
Colección	Dinámica	Estática
Flexibilidad	Menor	Mayor

getElementsByClassName()

```
let botones = document.getElementsByClassName("boton")
```

Busca específicamente elementos que tengan la clase "boton"

Además, devuelve una `HTMLCollection` viva, lo que significa que, si cambia el DOM, la colección puede actualizarse automáticamente

querySelectorAll()

```
let botones = document.querySelectorAll(".boton")
```

También encuentra los elementos con esa clase

Pero tiene una ventaja: permite utilizar selectores CSS más completos

Por ejemplo:

```
let botones = document.querySelectorAll("div .boton")
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Podemos buscar botones con clase boton que estén dentro de un div.

Además, `querySelectorAll()` devuelve una `NodeList` estática, por lo que los cambios posteriores en el DOM no modifican automáticamente esa lista

¿Cuándo utilizaría cada uno?

Usaría:

`getElementsByClassName()`

cuando solamente necesito buscar elementos por una clase y quiero algo sencillo

Usaría:

`querySelectorAll()`

cuando necesito hacer búsquedas más específicas utilizando selectores CSS

Solución 5

- `getElementById()` y `querySelector()`

Estos dos pueden hacer algo muy parecido.

Por ejemplo, podemos seleccionar: `<h1 id="titulo">Hola</h1>`

Con: `let titulo = document.getElementById("titulo")`

O con: `let titulo = document.querySelector("#titulo")`

Ambos pueden obtener el elemento h1.

Característica	<code>getElementById()</code>	<code>querySelector()</code>
Busca por	ID	Selector CSS
Resultado	Un elemento	Primer elemento
Si no encuentra	null	null
Flexibilidad	Menor	Mayor
Ejemplo	"titulo"	"#titulo"

`getElementById()`

Es específico para buscar por ID:

`let titulo = document.getElementById("titulo")`

`querySelector()`

Es más flexible porque puede buscar por ID, clase, etiqueta, atributos y otros selectores CSS:

`let titulo = document.querySelector("#titulo")`

También:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
let boton = document.querySelector(".boton")
```

¿Cuándo utilizaría cada uno?

Si necesito específicamente un elemento por su ID, podría utilizar:

```
getElementById()
```

porque es sencillo y directo

Si estoy trabajando con diferentes tipos de selectores y quiero utilizar la misma forma de búsqueda, usaría:

```
querySelector()
```

Actividad – 3 Investigación sobre selectores CSS

Enunciado

Investiga qué son los selectores CSS y cómo pueden utilizarse desde JavaScript.

Consulta específicamente:

- Selector por ID 1
- Selector por clase 2
- Selector por etiqueta 3
- Selector por atributo 4
- Selector descendiente 5
- Selector de elementos que tienen varias clases 6

Después explica:

¿Por qué `querySelector()` y `querySelectorAll()` permiten realizar selecciones más específicas?

Solución

¿Qué son los selectores CSS?

Los selectores CSS son formas de identificar y seleccionar elementos HTML según diferentes características. Aunque se utilizan principalmente en CSS para aplicar estilos, JavaScript también puede utilizarlos mediante `querySelector()` y `querySelectorAll()`

Selector por ID

Permite seleccionar un elemento utilizando su atributo id. Se identifica mediante el símbolo #. Normalmente se utiliza para seleccionar un elemento específico.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Selector por clase

Permite seleccionar elementos que tienen una determinada clase. Se identifica mediante el símbolo `..`. Una misma clase puede pertenecer a varios elementos

Selector por etiqueta

Permite seleccionar elementos según su etiqueta HTML, como párrafos, botones, divisiones o títulos. Es útil cuando queremos seleccionar elementos del mismo tipo

Selector por atributo

Permite seleccionar elementos que tienen un determinado atributo HTML. También puede utilizarse para buscar elementos que tengan un atributo con un valor específico

Selector descendiente

Permite seleccionar un elemento que se encuentra dentro de otro elemento. Se utiliza un espacio entre los selectores. Es útil para limitar la búsqueda a una determinada parte del documento

Selector de elementos que tienen varias clases

Permite seleccionar elementos que poseen dos o más clases al mismo tiempo. Las clases se escriben juntas, sin espacios

¿Por qué `querySelector()` y `querySelectorAll()` permiten selecciones más específicas?

Permiten realizar búsquedas más precisas porque podemos combinar diferentes características de los elementos HTML.

Actividad – 4 Laboratorio de experimentación

Actividad – 5 Preguntas de análisis

Enunciado

Responde con tus propias palabras:

1. ¿Cuál es la diferencia entre seleccionar por ID y seleccionar por clase?
2. ¿Qué método utilizarías cuando necesitas seleccionar muchos elementos?
3. ¿Qué diferencia encuentras entre `querySelector()` y `querySelectorAll()`?
4. ¿Qué método te parece más flexible? ¿Por qué?
5. ¿Qué método te parece más sencillo? ¿Por qué?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

6. ¿Todos los métodos devuelven el mismo tipo de resultado?
7. ¿Qué sucede si no existe el elemento que estás buscando?
8. ¿Por qué es importante conocer los selectores CSS para trabajar con JavaScript?
9. ¿En qué situación utilizarías `getElementsByTagName()`?
10. ¿En qué situación preferirías `querySelectorAll()`?

Solución

1. Seleccionar por ID sirve para encontrar un elemento específico, mientras que seleccionar por clase permite encontrar varios elementos que tienen la misma clase
2. Utilizaría `querySelectorAll()` porque permite seleccionar varios elementos al mismo tiempo
3. `querySelector()` encuentra el primer elemento que coincide con el selector, mientras que `querySelectorAll()` encuentra todos los elementos que coinciden
4. Me parece más flexible `querySelector()` porque permite utilizar diferentes tipos de selectores CSS, como ID, clases, etiquetas y combinaciones
5. Me parece más sencillo `getElementById()` porque solamente necesito escribir el ID del elemento que quiero encontrar
6. No. Algunos métodos devuelven un solo elemento y otros pueden devolver varios elementos
7. Si el elemento no existe, algunos métodos devuelven null y otros devuelven una colección vacía
8. Porque los selectores CSS permiten encontrar elementos de una forma más precisa y utilizar JavaScript para modificarlos o trabajar con ellos
9. Lo utilizaría cuando quisiera seleccionar todos los elementos que tengan una determinada etiqueta, por ejemplo, todos los elementos `<p>` o todos los `<button>`
10. Preferiría `querySelectorAll()` cuando necesitara seleccionar varios elementos utilizando una clase, una etiqueta o un selector CSS más específico

Actividad – 6 Reto investigativo

Enunciado

Un compañero afirma:

“Con `getElementById()` es suficiente para trabajar con el DOM, porque todos los elementos importantes pueden tener un ID.”

¿Estás de acuerdo?

Argumenta tu respuesta utilizando lo aprendido durante la investigación.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Solución

No estoy de acuerdo. `getElementById()` es útil para seleccionar un elemento específico, pero no es suficiente para trabajar con todo el DOM. En una página podemos tener varios elementos que compartan una misma clase o etiqueta, y en esos casos podemos utilizar `querySelectorAll()`. También podemos utilizar `querySelector()` para buscar elementos mediante diferentes selectores CSS. Por eso, conocer varios métodos permite seleccionar los elementos de una manera más flexible y adecuada según lo que necesitemos

Actividad – 7 Reto práctico